

Q&A; Chatbot with Groq

Complete Line-by-Line Code Explanation

Project	Q&A Chatbot with Groq LLM
File	app.py
Framework	Streamlit (Python Web App)
LLM Provider	Groq API (Fast AI Inference)
LLM Library	LangChain
Total Lines	~230 lines

SECTION 1 — Importing Libraries (Lines 1–6)

Code:

```
import streamlit as st
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
from langchain_groq import ChatGroq
import os
from dotenv import load_dotenv
```

Library	What It Does
streamlit	Creates the web UI — buttons, text boxes, sidebar, chat layout
StrOutputParser	Converts the LLM's response object into a plain readable string
ChatPromptTemplate	Creates a reusable template for how to ask questions to the AI
ChatGroq	Connects to Groq's super-fast AI servers to run LLM models
os	Reads system environment variables like API keys
load_dotenv	Loads secret keys from a .env file into the program

SECTION 2 — Loading Environment Variables (Lines 8–9)

Code:

```
load_dotenv()
```

This line reads the **.env** file in the project folder. The **.env** file stores secret API keys so they are not written directly in the code. For example, your **.env** file contains: **GROQ_API_KEY=your_key_here**. After calling `load_dotenv()`, the key becomes available via `os.getenv()`.

■ Best Practice: Never hardcode API keys in Python files. Always use **.env** files.

SECTION 3 — Streamlit Page Configuration (Lines 11–17)

Code:

```
st.set_page_config(
    page_title='Q&A Chatbot with Groq',
    page_icon='■',
    layout='wide',
    initial_sidebar_state='expanded'
)
```

Parameter	Value	Effect
page_title	Q&A Chatbot with Groq	Sets the browser tab title
page_icon	■	Sets the favicon/icon in browser tab
layout	wide	Uses full screen width instead of narrow column
initial_sidebar_state	expanded	Opens the sidebar automatically on load

■■ Important: `st.set_page_config()` MUST be the very first Streamlit command called. Placing any other `st.` command before it will cause an error.

SECTION 4 — Session State for Chat History (Lines 19–20)

Code:

```
if 'chat_history' not in st.session_state:
    st.session_state.chat_history = []
```

Streamlit re-runs the entire script from top to bottom every time a user clicks a button or types something. Without `session_state`, all variables would reset to empty on each re-run, losing the chat history.

`st.session_state` acts like a persistent memory across re-runs. This code checks: 'Does `chat_history` already exist?' — if not, create it as an empty list. Each conversation (question + answer) gets stored as a tuple in this list.

SECTION 5 — LangSmith Tracking (Lines 22–24)

Code:

```
if os.getenv('LANGCHAIN_API_KEY'):
    os.environ['LANGCHAIN_TRACING_V2'] = 'true'
    os.environ['LANGCHAIN_PROJECT'] = os.getenv('LANGCHAIN_PROJECT', 'Simple Q&A Chatbot With Groq')
```

LangSmith is an optional monitoring/debugging tool by LangChain. If you have a LangSmith API key in your `.env` file, it will log all LLM calls (inputs, outputs, latency) to the LangSmith dashboard for debugging. This block only activates if the key exists — it is completely optional.

SECTION 6 — Prompt Template (Lines 26–29)

Code:

```
prompt = ChatPromptTemplate.from_messages([
    ('system', 'You are a helpful assistant. Please respond to the user queries in a clear and concise manner.'),
    ('user', 'Question: {question}')
```

```
])
```

ChatPromptTemplate creates a structured message format for the LLM. It has two parts:

system message: Sets the AI's persona and behaviour. This tells the AI to act as a helpful assistant.

user message: The actual question from the user. The {question} is a placeholder that gets filled with the real question at runtime.

■ Think of it like a letter template: the structure is fixed, but {question} gets replaced with whatever the user types.

SECTION 7 — generate_response() Function (Lines 31–57)

Code:

```
def generate_response(question, model_name, temperature, max_tokens, api_key):
    try:
        llm = ChatGroq(groq_api_key=api_key, model_name=model_name, ...)
        output_parser = StrOutputParser()
        chain = prompt | llm | output_parser
        answer = chain.invoke({'question': question})
        return answer, None
    except Exception as e:
        return None, str(e)
```

Parameters explained:

Parameter	Purpose
question	The text question typed by the user
model_name	Which Groq AI model to use (e.g. llama-3.3-70b)
temperature	Controls creativity: 0.0 = precise, 1.0 = creative/random
max_tokens	Maximum word-length of the AI's response
api_key	The Groq API key to authenticate the request

The LangChain Pipeline (chain):

```
chain = prompt | llm | output_parser
```

This is LangChain's LCEL (LangChain Expression Language) pipeline using the | (pipe) operator. It works like an assembly line:

Step	Component	What Happens
1	prompt	Takes the user's question, fills in the {question} placeholder
2	llm (ChatGroq)	Sends the formatted prompt to Groq's AI server, gets AI response
3	output_parser	Converts the AI response object into a simple plain string

SECTION 8 — main() Function: UI Layout (Lines 59–200)

The main() function builds the entire visual interface using Streamlit components. It is divided into three areas: Title, Sidebar, and Chat Interface.

8A — Title Area

```
st.title('■ Enhanced Q&A Chatbot with Groq')
st.markdown('Ask me anything! ...')
```

`st.title()` renders a large heading. `st.markdown()` renders formatted text using Markdown syntax.

8B — Sidebar (API Key + Settings)

```
with st.sidebar:
    groq_api_key = os.getenv('GROQ_API_KEY')
    if not groq_api_key:
        groq_api_key = st.text_input('Enter your Groq API Key:', type='password')
```

The sidebar contains all configuration controls. First it tries to load the API key from the `.env` file. If no key is found there, it shows a password input box for the user to type it manually.

8C — Model Selection Dropdown

```
model_name = st.selectbox('Select Groq Model', ['llama-3.3-70b-versatile', ...])
```

Shows a dropdown with 5 available Groq models. Each model has different speed/capability tradeoffs. The selected model name is passed to ChatGroq when generating a response.

Model	Best For
llama-3.3-70b-versatile	Complex questions, most capable
llama-3.1-8b-instant	Fast simple queries
mixtral-8x7b-32768	Long documents (32K context)
gemma2-9b-it	Efficient, instruction-following tasks

8D — Temperature and Max Tokens Sliders

```
temperature = st.slider('Temperature', min_value=0.0, max_value=1.0, value=0.7)
max_tokens = st.slider('Max Tokens', min_value=50, max_value=2048, value=512)
```

Temperature: Controls randomness of AI output. 0.0 = always gives the same, factual answer. 1.0 = creative, varied, sometimes unpredictable answers. 0.7 is the default — a good balance.

Max Tokens: Limits how long the response can be. 1 token $\approx$ 0.75 words. 512 tokens $\approx$ ~384 words.

8E — Clear Chat Button

```
if st.button('■ Clear Chat History', use_container_width=True):
    st.session_state.chat_history = []
    st.rerun()
```

When clicked, it empties the `chat_history` list in `session_state`, then calls `st.rerun()` to refresh the page — showing a clean, empty chat.

SECTION 9 — Displaying Chat History (Lines 170–185)

```
for i, (question, answer) in enumerate(st.session_state.chat_history):
    col1, col2 = st.columns([1, 1])
    with col1: st.info(f'You: {question}')
```

```
with col2: st.success(f'Bot: {answer}')
```

Loops through every saved question-answer pair. Displays them side-by-side using two equal columns. `st.info()` shows a blue box for user questions. `st.success()` shows a green box for bot answers.

SECTION 10 — Question Input Form (Lines 187–215)

```
with st.form(key='question_form', clear_on_submit=True):
    user_input = st.text_input('Your Question:')
    submit_button = st.form_submit_button('■ Send')
```

`st.form()` groups the text input and submit button together. `clear_on_submit=True` automatically clears the input box after the user clicks Send. This is better than a standalone button because it also supports pressing Enter to submit.

Processing the Submission:

```
if submit_button and user_input:
    with st.spinner('■ Thinking...'):
        response, error = generate_response(...)
        st.session_state.chat_history.append((user_input, response))
    st.rerun()
```

Only runs when both conditions are true: button was clicked AND the input is not empty. `st.spinner()` shows a loading animation while the AI is generating. After getting the response, it saves both question and answer as a tuple into `chat_history`, then calls `st.rerun()` to refresh and display the new message.

SECTION 11 — Entry Point (Last 2 Lines)

```
if __name__ == '__main__':
    main()
```

This is standard Python practice. It means: only run `main()` if this file is executed directly (not imported as a module). When you run 'streamlit run app.py', Streamlit executes this file directly, so `main()` gets called and the app launches.

SECTION 12 — Complete App Flow Summary

Step	What Happens
1. Startup	<code>load_dotenv()</code> reads <code>.env</code> , page config sets up the browser tab
2. First Load	<code>session_state</code> initializes empty <code>chat_history</code> list
3. UI Renders	Sidebar shows config options, main area shows empty chat
4. User Types	User types question in text box and clicks Send
5. API Call	<code>generate_response()</code> builds prompt → sends to Groq → gets answer
6. Save & Show	Answer saved to <code>session_state</code> , <code>st.rerun()</code> refreshes page
7. Display	Chat history loop shows all Q&A pairs on screen
8. Repeat	User can ask more questions; history grows each time

